

Mobile Application Development

Algorithms & Code Snippets

Made by: Abdullah Javed

Intent:

Let's have a look at intents with the help of an example. Lets say we want to switch from one activity to another with the tap of a button. We can do it the following way:

- Create an `onClickListener()` for that specific button
- Inside the listener, create an intent
- Give it a suitable name and provide it with the necessary parameters like the source activity and destination activity
- Start the intent
- Done

Snippet:

```
myButton.setOnClickListener() {  
    Intent myIntent = new Intent(this, AnotherActivity.class)  
    startActivity(myIntent);  
}
```

If you want to send a message with the intent, you can do that as well

```
myButton.setOnClickListener() {  
    Intent myIntent = new Intent(this, AnotherActivity.class)  
    myIntent.putExtra(AnotherActivity.EXTRA_MESSAGE, "A Message");  
    startActivity(myIntent);  
}
```

ListView Using Array Adapter:

A simple list view is populated and displayed with the help of ArrayAdapter. Remember, if your list has each of its items as a simple string, only then you can use this method, if your list is even a little more complex, you have to use BaseAdapter. Lets have a look at how we create, populate and display ListViews:

- Initialize your listview and array of string data
- Create an adapter
- Provide the adapter with your activity context, layout of your list items and your array that you initialized previously
- Provide your listview with the id of your actual listview from your layout file
- Set the adapter
- Done

Snippet:

```
String[] myData = {.....}
```

```
ListView myListView;
```

```
ArrayAdapter myArrayAdapter = new ArrayAdapter<String>(this, list_item_layout, myData)
```

```
myListView = findViewById(R.id.list_view_id)
```

```
myListView.setAdapter(myArrayAdapter)
```

Base Adapter:

Now lets have a look at how we can use a more complex ListView with the help of a Base Adapter, which is made just for the purpose.

- Create a listview in your layout file
- Create another layout file in which you will design how each list item looks and what does it contain
- Create a CustomeBaseAdapter class

- Implement the four main methods of that class
- Declare all the data which will be present inside each of your items
- Inside the getView function, use the inflater to inflate the item layouts
- Assign the id's of all the fields from the item layout file to the above declared data fields
- Return the inflater
- Create a listView variable in the main file
- Set the adapter to that listView variable, but this time, instead of using a simple array adapter, use your own custom base adapter which you created in a separate file in the previous steps.
- Done

Snippet:

```
public class CustomBaseAdapter extends BaseAdapter {

    Context context;
    String[] OSName;
    int[] OSImageLoc;
    LayoutInflater inflater;

    public CustomBaseAdapter(Context context, String[] OSName, int[] OSImageLoc){
        this.context = context;
        this.OSName = OSName;
        this.OSImageLoc = OSImageLoc;
        inflater = LayoutInflater.from(context);
    }

    @Override
    public int getCount() {
        return OSName.length;
    }

    @Override
    public Object getItem(int position) {
        return OSName[position];
    }

    @Override
    public long getItemId(int position) {
        return position;
    }

    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
        // get view of custom item layout
        convertView = inflater.inflate(R.layout.custom_list_view_item, null);

        ImageView OSImageView = (ImageView) convertView.findViewById(R.id.list_item_image);
        TextView OSText = (TextView) convertView.findViewById(R.id.list_item_text);

        OSImageView.setImageResource(OSImageLoc[position]);
        OSText.setText(OSName[position]);
        return convertView;
    }
}

operatingSystemList = (ListView) findViewById(R.id.mobile_list);
operatingSystemList.setAdapter(new CustomBaseAdapter(getApplicationContext(), OSName, OSImageLoc));
```

Shared Preferences:

Shared preferences are a simple and easy way to save data permanently in the app. However it is only used for a very small amount of data and using it for larger amounts of data is not a good practice. Lets have a look at how we can use shared preferences:

- Create a separate java class/file for the shared preferences
- Create keys for your data. For instance if you have to store a name, you just have to create a string like so: **private static final MY_NAME_KEY = "name";**
- The next step is to create methods like saving the data, retrieving and so on.
- Save the data using putString() function.
- Retrieve data using getString() function
- Always use editor.commit() after making any changes.
- In the main file, use the shared preferences
- If you are fetching a name from a textview, store it in a variable
- Provide the name to your sharedpreferences class
- Done

Snippet:

```
public class MySharedPrefs {
    public static final String USER_NAME = "userName";
    public static final String PASS_WORD = "passWord";

    public static boolean setUserNamePW(Context context, String userName, String passWord){
        SharedPreferences prefs = context.getSharedPreferences("MyPrefs", context.MODE_PRIVATE);
        SharedPreferences.Editor editor = prefs.edit();

        editor.putString(USER_NAME, userName);
        editor.putString(PASS_WORD, passWord);
        return editor.commit();
    }

    public static String[] getUserNamePW(Context context){
        SharedPreferences prefs = context.getSharedPreferences("MyPrefs", context.MODE_PRIVATE);

        String[] userNamePW = new String[2];
        userNamePW[0] = prefs.getString(USER_NAME, "");
        userNamePW[1] = prefs.getString(PASS_WORD, "");
        return userNamePW;
    }

    public static boolean removeUserNamePW(Context context){
        SharedPreferences prefs = context.getSharedPreferences("MyPrefs", context.MODE_PRIVATE);
        SharedPreferences.Editor editor = prefs.edit();

        editor.remove(USER_NAME);
        editor.remove(PASS_WORD);
        return editor.commit();
    }
}
```

```

public void saveButton(View view){
    String userNameStr = userName.getText().toString();
    String passWordStr = password.getText().toString();
    if(!userNameStr.isEmpty() &&
        !passWordStr.isEmpty()){
        if(MySharedPrefs.setUserNamePW(this, userNameStr, passWordStr){
            Toast.makeText(this, "Successfully Saved", Toast.LENGTH_SHORT).show();
        } else {
            Toast.makeText(this, "Not Successfully Saved", Toast.LENGTH_SHORT).show();
        }
    }
}

public void restoreButton(View view){
    String[] userNamePW = MySharedPrefs.getUserNamePW(this);
    if(!userNamePW[0].isEmpty() && !userNamePW[0].isEmpty()){
        userName.setText(userNamePW[0]);
        password.setText(userNamePW[1]);
    }
}

public void deleteButton(View view){
    if(MySharedPrefs.removeUserNamePW(this){
        userName.setText("");
        password.setText("");
        Toast.makeText(this, "Successfully Deleted", Toast.LENGTH_SHORT).show();
    } else {
        Toast.makeText(this, "Not Successfully Deleted", Toast.LENGTH_SHORT).show();
    }
}

```

Fragments:

Fragments are a lightweight part of an app which can replace any unnecessary activities in the app because the more the number of activities in an app, the more resource hungry it becomes. We just create one activity and then add as many fragments in it as we want, this saves us memory without compromising on functionality at all. Lets take a look at the steps involved in doing so:

- Create a separate java file for each fragment you are making
- All the functionality that your activity does will now be done inside the fragments
- Create separate layout files for each fragment
- Declare your fragments in the main file
- Use a button to switch between the fragments
- Inside the clickListener, create the logic to switch between fragments
- Get the fragmentManager
- beginTransaction
- use the replace method, and provide it with the fragment name you want to replace

- commit
- done

Snippet:

```

fragmentA = new FragmentA();
fragmentB = new FragmentB();

replaceFragment(fragmentA);

Button replaceBtn = findViewById(R.id.replace_btn);
replaceBtn.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        if(!isFragmentA) {
            replaceFragment(fragmentA);
            isFragmentA = true;
        } else {
            replaceFragment(fragmentB);
            isFragmentA = false;
        }
    }
});

void replaceFragment(Fragment fragment){
    fragmentManager = getSupportFragmentManager();
    fragmentTransaction = fragmentManager.beginTransaction();
    fragmentTransaction.replace(R.id.fragment_container, fragment);
    fragmentTransaction.addToBackStack(null);
    fragmentTransaction.commit();
}

```

Fragments with ViewPager:

ViewPager is used to set the fragments in a tabular layout, as in whatsapp main screen which changes the fragments upon swiping left or right.

- Create the fragments and their layouts as before
- Create a PagerAdapter file
- Make 3 functions (getItem, getCount, getPageTitle)
- In the getItem, return all your fragments one by one

```

public class MyPagerAdapter extends FragmentPagerAdapter {
    public MyPagerAdapter(FragmentManager fm) {
        super(fm);
    }

    @Override
    public Fragment getItem(int position) {
        switch(position){
            case 0:
                return new FirstFragment();
            case 1:
                return new SecondFragment();
            case 2:
                return new ThirdFragment();
        }
        return null;
    }
}

```

- In the getCount, return a number equal to the number of fragments

```

@Override
public int getCount() {
    return 3; // since we have three fragments
}

```

-
- In the getPageTitle, return the titles for each fragment

```

@Nullable
@Override
public CharSequence getPageTitle(int position) {

    switch(position) {
        case 0:
            return "First Tab";
        case 1:
            return "Second Tab";
        case 2:
            return "Third Tab";
    }

    return null;
}

```

-
- In the main file, use your custom pager that you just made.
- Get fragment support manager
- Initialize the viewpager ui component
- Initialize the tab layout
- Set the viewpager object with the tab method
- Set the adapter for viewpager using your own custom page adapter

```

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        MyPagerAdapter pageAdapter = new MyPagerAdapter(getSupportFragmentManager());
        ViewPager viewPager = findViewById(R.id.view_pager);

        TabLayout tab = findViewById(R.id.tabs);
        tab.setupWithViewPager(viewPager);

        viewPager.setAdapter(pageAdapter);
    }
}

```

- 1

SQLite:

- We will create two separate files, one will be our contract class and the other will be our db helper class
- Contract class will define the structure of our table and some queries related to it
- Create the class
- Define the table name, columns and queries such as create table, delete table, select all etc.

```
public final class SchoolContract {  
  
    public static class Student {  
        // Tables and Column names  
        public static final String TABLE_NAME = "student";  
        public static final String COLUMN_ID = "id";  
        public static final String COLUMN_NAME = "name";  
        public static final String COLUMN_ADDRESS = "address";  
  
        // SQL queries  
        public static final String CREATE_STUDENT_TABLE =  
            "CREATE TABLE " + TABLE_NAME + "(" +  
                COLUMN_ID + " INTEGER PRIMARY KEY, " +  
                COLUMN_NAME + " TEXT, " +  
                COLUMN_ADDRESS + " TEXT) ";  
  
        public static final String DELETE_STUDENT_TABLE =  
            "DROP TABLE IF EXISTS " + TABLE_NAME;  
  
        public static final String READ_ALL_TABLE = "SELECT * FROM " + TABLE_NAME;  
    }  
}
```

-
- Create a db helper class
- Provide the create table query from the contract class to the onCreate method inside the helper class

```
public class SchoolDBHelper extends SQLiteOpenHelper {  
    Context context;  
    public static final int DATABASE_VERSION = 1;  
    public static final String DATABASE_NAME = "School.db";  
  
    public SchoolDBHelper(@Nullable Context context) {  
        super(context, DATABASE_NAME, null, DATABASE_VERSION);  
        this.context = context;  
    }  
  
    @Override  
    public void onCreate(SQLiteDatabase db) {  
        db.execSQL(SchoolContract.Student.CREATE_STUDENT_TABLE);  
    }  
  
    @Override  
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {  
        // This database is only a cache for online data, so its upgrade policy is  
        // to simply to discard the data and start over  
        db.execSQL(SchoolContract.Student.DELETE_STUDENT_TABLE);  
        onCreate(db);  
    }  
}
```

-
- Create methods inside the helper class for adding, deleting, editing or getting data based on your requirements

```
// add id, name, address to table
public void addEntry(int id, String name, String address){
    SQLiteDatabase db = getWritableDatabase();
    // Create a new map of values, where column names are the keys
    ContentValues values = new ContentValues();
    values.put(SchoolContract.Student.COLUMN_ID, id);
    values.put(SchoolContract.Student.COLUMN_NAME, name);
    values.put(SchoolContract.Student.COLUMN_ADDRESS, address);
    try{
        // Insert the new row
        db.insert(SchoolContract.Student.TABLE_NAME, null, values);
    }catch (Exception e){
        Toast.makeText(context, "Error: " + e, Toast.LENGTH_SHORT).show();
    }
    db.close();
}
```

-
- Create a setter class to make the data usable for the frontend
- Create getter functions to get the desired data

```
public class SchoolSetter {
    int id;
    String name;
    String address;

    public SchoolSetter(int id, String name, String address) {
        this.id = id;
        this.name = name;
        this.address = address;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public String getAddress() {
        return address;
    }
}
```

-
- Create a base adapter for the listview
- Call the getter functions from setter class to set the data in the adapter

```
public class SchoolBaseAdapter extends BaseAdapter {
    ArrayList<SchoolSetter> setter;
    LayoutInflater inflater;
    public SchoolBaseAdapter(Context context, ArrayList<SchoolSetter> setter) {
        this.setter = setter;
        inflater = LayoutInflater.from(context);
    }

    @Override
    public int getCount() {
        return setter.size();
    }

    @Override
    public Object getItem(int position) {
        return setter.get(position);
    }

    @Override
    public long getItemId(int position) {
        return position;
    }

    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
        // get view of custom item layout
        convertView = inflater.inflate(R.layout.list_view_item, null);
        TextView id = convertView.findViewById(R.id.item_id);
        TextView name = convertView.findViewById(R.id.item_name);
        TextView address = convertView.findViewById(R.id.item_address);

        id.setText(String.valueOf(setter.get(position).getId()));
        name.setText(String.valueOf(setter.get(position).getName()));
        address.setText(String.valueOf(setter.get(position).getAddress()));
        return convertView;
    }
}
```

-
- In the main activity, use the adapter to set the listview for the initial data

```

public class MainActivity extends AppCompatActivity {
    EditText idET, nameET, addressET;
    SchoolDBHelper dbHelper;
    ListView listView;
    SchoolBaseAdapter adapter;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        idET = findViewById(R.id.id_edit_box);
        nameET = findViewById(R.id.name_edit_box);
        addressET = findViewById(R.id.addr_edit_box);

        listView = findViewById(R.id.dbListView);
        // create DB
        dbHelper = new SchoolDBHelper(getApplicationContext());

        ArrayList<SchoolSetter> schoolSetters = dbHelper.readAll();
        adapter = new SchoolBaseAdapter(getApplicationContext(), schoolSetters);

        listView.setAdapter(adapter);
    }
}

```

-
- Create onupdate methods to update the list after a new entry is made

```

public void onUpdateDB(View view){
    int id = Integer.valueOf(idET.getText().toString());
    String name = nameET.getText().toString();
    String address = addressET.getText().toString();
    dbHelper.addEntry(id, name, address);
    updateListView();
}

void updateListView(){
    ArrayList<SchoolSetter> schoolSetters = dbHelper.readAll();
    adapter = new SchoolBaseAdapter(getApplicationContext(), schoolSetters);
    listView.setAdapter(adapter);
    listView.invalidateViews();
}

```

-

Android Service:

- Create a custom service class
- Inside the oncreatecommand method, create and start a new thread

```

public class MyService extends Service {
    @Nullable
    @Override
    public IBinder onBind(Intent intent) {
        return null;
    }
    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        Log.d("MyService", "onStartCommand: Service Started");

        serviceThread thread = new serviceThread(startId);
        thread.start();

        return START_STICKY;
    }
}

```

-
- Create a thread class inside the service class and declare an integer

```

class serviceThread extends Thread {
    int threadID;

    serviceThread(int threadID){
        this.threadID = threadID;
    }
}

```

-
- Create a run function, and perform all the functionality of the service inside the run function

```

@Override
public void run() {
    try {
        for (int i = 0; i < 10; i++) {
            Log.d("MyService", "serviceThread Running i = " + i);
            Thread.sleep(1000);
        }
    } catch (Exception e) {
        Log.d("MyService", "serviceThread Error " + e);
    }
    stopSelf(threadID);
}

```

-
- Use the service in the main activity through intent
- Use startService and stopService methods

```

public class MainActivity extends AppCompatActivity {

    Intent serviceIntent;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main_service);
    }

    public void startService(View view){
        serviceIntent = new Intent(getApplicationContext(), MyService.class);
        startService(serviceIntent);
    }

    public void stopService(View view){
        serviceIntent = new Intent(getApplicationContext(), MyService.class);
        stopService(serviceIntent);
    }
}

```

-

Volley Networking:

- In the main file, start a new request queue


```

public void onClick(View v) {
    // Instantiate the RequestQueue.
    RequestQueue queue = Volley.newRequestQueue(getApplicationContext());
    //Start a new GET request for the url "https://www.york.ac.uk/teaching/cws/www/webpage1.html"

```
-
- Get the url from an edit text or a string etc.


```

//String url = "https://www.york.ac.uk/teaching/cws/www/webpage1.html";
String url = myEditText.getText().toString();

```
-
- Request a string response from the url
- Call the onResponse method to provide the response to the desired ui element (webview in our case)

- ```
StringRequest stringRequest = new StringRequest(Request.Method.GET, url,
 new Response.Listener<String>() {
 @Override
 public void onResponse(String response) {
 Log.d(tag: "Main", msg: "Response is" + response);
 myWebView.loadUrl(url);
 }
 })
```

## Broadcast Receiver:

For System generated intents:

- Create a new receiver using MyReceiver class
- Use Intent filter to provide the type of pre-defined intent you want to use
- Register the receiver

```
MyReceiver receiver = new MyReceiver();
IntentFilter filter = new IntentFilter("android.intent.action.AIRPLANE_MODE");
registerReceiver(receiver,filter);
```

- 

For Custom Broadcasts:

- Create a class for the receiver extending BroadcastReceiver class
- Create the logic of what the broadcast receiver is going to do (inside the onReceive method)

```
public class MyReceiver extends BroadcastReceiver {
 @Override
 public void onReceive(Context context, Intent intent) {
 Toast.makeText(context, "Intent Detected: " + intent.getAction(), Toast.LENGTH_LONG).show();
 }
}
```

- 
- Use the receiver in the main class inside the broadcastIntent method
- Create a new intent
- Use setAction method to specify the broadcast receiver class
- Send the intent using sendBroadcast method.

```
public void broadcastIntent(View view) {
 Intent intent = new Intent();
 intent.setAction("com.example.mybroadcastreceiver.CUSTOM_INTENT");
 sendBroadcast(intent);
}
```

-